SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

PROJECT TITLE

PROPERTY RENTAL AND LEASE MANAGEMENT SYSTEM

Program	CSE AIML
Section	Rhinos
Faculty	Dr. Sahithi
Date of Submission	10/10/2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0101090	Pakanati Priyakaran Reddy	9703963456	priyakaran.p_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Dr. Sahithi, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled **“Property Rental and Lease Management System”**, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date	**:**	**10/10/2026**	**Name:**	**Pakanati Priyakaran Reddy**
Place	**:**	**Hyderabad**	**Roll Number:**	**25WU0101090**
			Program:	**CSE AIML**

TABLE OF CONTENTS

Contents	Page No.
Abstract	7
LIST OF TABLES	5
LIST OF FIGURES	5
LIST OF ACRONYMS	6
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	8
1.1 Problem Statement	8
1.2 Objectives	8
1.3 Scope and Assumptions	8
1.4 Proposed Solution Strategy	9
1.5 Functional Requirements	9
1.6 Entity-Relationship Diagram (ERD)	10
1.7 Entity and Relationship Summary	10
2. LOGICAL DATABASE DESIGN	12
2.1 Relational Schema	12
2.2 Keys and Integrity Constraints	12
2.3 Functional Dependencies	13
2.4 Normalization up to Third Normal Form (3NF)	13
2.5 Data Dictionary	14
2.6 Implementation of Logical Design: DDL and Sample Data	15
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	16
3.1 Application Overview	16
3.2 Development Environment and Database Connectivity	16
3.3 Application Modules and Database Mapping	16
3.4 CRUD Operations	17
3.5 Business Rules, Validation and Error Handling	17
3.6 Reports and Database Operations	18
3.7 Application Screens	18
3.8 Testing and Results	21
4. CONCLUSION	22
4.1 Limitations and Future Enhancements	22
5. REFERENCES	22
6. APPENDICES	23

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	9
Table 2	Proposed Solution Strategy	9
Table 3	Functional Requirements	9
Table 4	Entity and Relationship Summary	10
Table 5	Relational Schema	12
Table 6	Keys and Integrity Constraints	12
Table 7	Normalization Status up to 3NF	13
Table 8	Data Dictionary	14
Table 9	Database Implementation Environment	15
Table 10	Application Development Environment	16
Table 11	Application Module-to-Database Mapping	16
Table 12	CRUD Operations	17
Table 13	Business Rules, Validation and Error Handling	17
Table 14	Reports and Database Operations	18
Table 15	Test Cases	21

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	10
Figure 2	Dashboard: Occupancy, Revenue and Overdue-Rent Reports	19
Figure 3	Add Property: Data Entry Screen	19
Figure 4	Search: Tenant Search Results	19
Figure 5	Edit Owner: Update Operation	20
Figure 6	Rent Payment Transaction	20
Figure 7	Create Lease: Validation of Lease Dates	20
Figure 8	Referential Integrity: Blocked Deletion	21

LIST OF ACRONYMS

Acronym	Expansion
3NF	Third Normal Form
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
INR	Indian Rupee
PBL	Project-Based Learning
PK	Primary Key
SQL	Structured Query Language
UI	User Interface

ABSTRACT

Rental property owners and small agencies usually track properties, tenants, leases and rent payments in paper registers or spreadsheets, which leads to duplicate records, missed lease renewals, payment-tracking errors and slow retrieval of tenant or property history. The objective of this project is to design and implement a database-backed application that centralizes property, owner, tenant, lease, payment and maintenance records for a rental management business. An Entity-Relationship model of six entities was converted into a relational schema normalized up to Third Normal Form and implemented in MySQL with primary keys, foreign keys and CHECK constraints. A Python application, built with Flask and the mysql-connector-python driver, provides create, read, search, update and controlled delete operations for every module, enforces business rules such as valid lease date ranges and referential integrity, and generates occupancy, revenue and overdue-rent reports directly from the database. Eight positive and negative test cases confirmed that the application correctly uses the designed database and enforces the intended rules. The completed system demonstrates a practical, rule-enforcing alternative to manual rental record-keeping.

KEYWORDS

Keywords: Relational Database, Property Rental Management, ER Diagram, Normalization, MySQL, Python

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Independent property owners and small rental agencies commonly manage their portfolio of rental properties using paper registers or disconnected spreadsheets. Under this manual approach, property details, tenant information, lease terms and rent payments are recorded separately, which makes it difficult to verify whether a property is currently occupied, whether a tenant's rent is paid for the month, or when a lease is due for renewal. Owners, property managers and tenants are all affected: owners struggle to track income and occupancy, managers spend significant time reconciling records by hand, and tenants have no reliable record of their payment history.

The lack of a centralized, rule-enforcing system results in duplicate or inconsistent records, missed lease renewals, undetected payment delays and slow retrieval of historical data. This project addresses these limitations by designing and implementing a relational database-backed application, the Property Rental and Lease Management System, that stores all property, tenant, lease, payment and maintenance data in a single normalized database and enforces the business rules needed to keep that data accurate.

1.2 Objectives

- Design a normalized relational database (up to 3NF) that models owners, properties, tenants, leases, payments and maintenance requests.
- Develop a Python application that connects to the database and provides data-entry, search and viewing screens for every module.
- Implement create, read, update and controlled delete operations that enforce primary key, foreign key and referential integrity constraints.
- Validate business rules such as lease date ranges, mandatory fields and blocked deletion of referenced records.
- Generate occupancy, revenue and overdue-payment reports directly from the database using SQL aggregate queries.

1.3 Scope and Assumptions

The scope of the system and the assumptions made during design are summarized in Table 1.

Table 1. Scope and Assumptions

Item	Description
In Scope	Property, owner, tenant, lease, payment and maintenance-request management; occupancy and revenue reporting; data validation and referential integrity enforcement.
Out of Scope / Exclusions	Online rent payment gateway integration, mobile application, multi-branch or multi-currency support, and tenant-facing self-service login.
Assumptions	A single administrative user (property manager) operates the application; all amounts are in INR; each property is associated with exactly one owner; a property has at most one active lease at a time.

1.4 Proposed Solution Strategy

The proposed solution replaces manual registers with a single database accessed through a web application. Table 2 describes the intended users, modules, workflow, business rules and outputs.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Property manager / administrator operating the web application.
Major Application Modules	Owners, Properties, Tenants, Leases, Payments, Maintenance Requests and a Reports dashboard.
Proposed Workflow	An owner is registered, properties are added and linked to the owner, a tenant is registered, a lease links a tenant to a property, rent payments are recorded against the lease, and maintenance requests are logged against the property and tenant.
Important Business Rules	A lease's end date must be after its start date; a record cannot be deleted while another record still references it (e.g., a property with a lease); a property's occupancy status is derived from whether an active lease currently covers it.
Expected Reports / Outputs	Occupancy report by property type, monthly revenue summary, and a list of leases with rent overdue for the current month.

1.5 Functional Requirements

The functional requirements that the application must satisfy are listed in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The system shall allow adding, viewing, searching, editing and removing owner records.	High
FR-02	The system shall allow adding, viewing, searching, editing and removing property records linked to an owner.	High
FR-03	The system shall allow adding, viewing, searching, editing and removing tenant records.	High
FR-04	The system shall allow creating and editing a lease linking a tenant to a property, rejecting an end date on or before the start date.	High
FR-05	The system shall allow recording rent payments against a lease.	High
FR-06	The system shall allow logging, viewing and updating maintenance requests against a property and tenant.	Medium
FR-07	The system shall block deletion of any record that is still referenced by another record.	High
FR-08	The system shall generate occupancy, revenue and overdue-payment reports from the live database.	High

Req. ID	Functional Requirement	Remarks / Priority
FR-09	The system shall allow searching the records of every module by any displayed value.	Medium

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the approved ER diagram. It identifies six entities (Owner, Property, Tenant, Lease, Payment and Maintenance), their attributes and primary keys, and the one-to-many relationships that connect them. Every foreign key is mandatory, so each child record must reference an existing parent record.

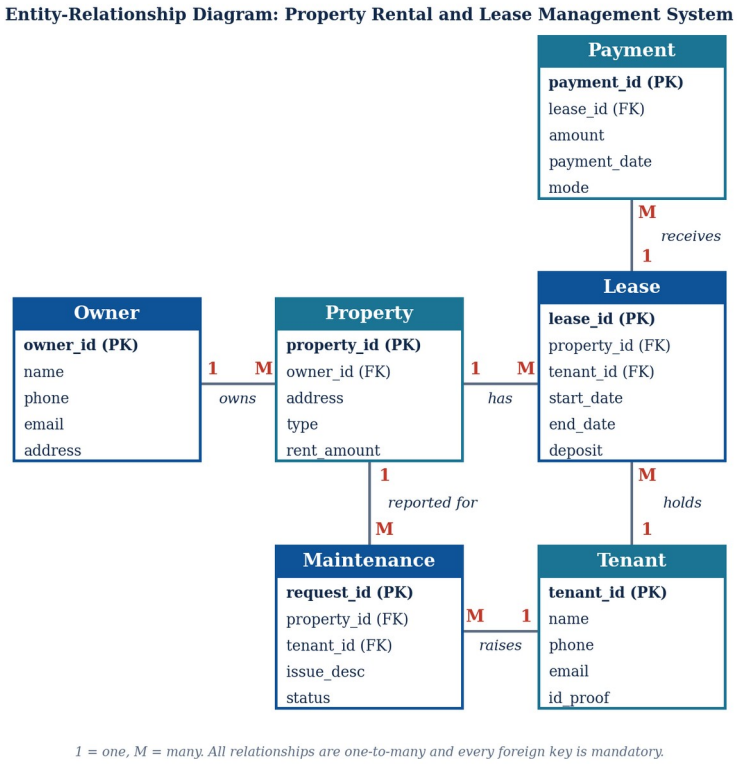


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

The entities, their major attributes, cardinalities and purposes are summarized in Table 4.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Owner	owner_id (PK), name, phone, email, address	Stores the property owner's contact details.
Property	property_id (PK), owner_id (FK), address, type, rent_amount; Owner 1:M Property	Stores each rental unit and links it to its owner.
Tenant	tenant_id (PK), name, phone, email, id_proof	Stores tenant contact and identification details.
Lease	lease_id (PK), property_id (FK), tenant_id (FK), start_date, end_date, deposit; Property 1:M Lease, Tenant 1:M Lease	Links a tenant to a property for a defined rental period.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Payment	payment_id (PK), lease_id (FK), amount, payment_date, mode; Lease 1:M Payment	Records rent payments made against a lease.
Maintenance	request_id (PK), property_id (FK), tenant_id (FK), issue_desc, status; Property 1:M Maintenance, Tenant 1:M Maintenance	Tracks maintenance issues reported for a property.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The approved ER model was converted into six relations by mapping each entity to a table with a surrogate primary key and each one-to-many relationship to a foreign key on the "many" side, following the standard ER-to-relational mapping rules [1]. Table 5 lists each table and its purpose; the column-level design is given in the data dictionary (Section 2.5).

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	owner	Stores property owner records.
2	property	Stores rental property/unit records, linked to an owner.
3	tenant	Stores tenant records.
4	lease	Stores lease agreements linking a tenant to a property.
5	payment	Stores rent payments made against a lease.
6	maintenance	Stores maintenance requests linked to a property and tenant.

2.2 Keys and Integrity Constraints

Primary keys, foreign keys and the other integrity constraints enforced by the DBMS [2] are listed in Table 6. All foreign keys use the default RESTRICT behaviour, so a parent record cannot be deleted while child records exist.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
owner	Primary Key	owner_id	Uniquely identifies each owner.
	Not Null	name, phone	Core contact details are mandatory.
property	Primary Key	property_id	Uniquely identifies each property.
	Foreign Key	owner_id → owner	Every property must belong to an existing owner.
	Not Null	owner_id, address, type, rent_amount	Basic property details are mandatory.
	Check	rent_amount > 0; type in (Apartment, House, Shop, Office)	Rent must be positive and the type must be valid.
tenant	Primary Key	tenant_id	Uniquely identifies each tenant.
	Not Null	name, phone	Core contact details are mandatory.
lease	Primary Key	lease_id	Uniquely identifies each lease.
	Foreign Key	property_id → property, tenant_id → tenant	A lease must reference an existing property and tenant.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
	Not Null	property_id, tenant_id, start_date, end_date	A lease must have parties and dates.
	Check	end_date > start_date; deposit > 0	A lease must span a valid date range with a positive deposit.
payment	Primary Key	payment_id	Uniquely identifies each payment.
	Foreign Key	lease_id → lease	A payment must reference an existing lease.
	Not Null	lease_id, amount, payment_date, mode	Every payment must be fully described.
	Check	amount > 0; mode in (Cash, UPI, Bank Transfer, Cheque)	Payment amount must be positive and the mode valid.
maintenance	Primary Key	request_id	Uniquely identifies each request.
	Foreign Key	property_id → property, tenant_id → tenant	A request must reference an existing property and tenant.
	Not Null	property_id, tenant_id, issue_desc, status	A request must describe the issue.
	Default	status = 'Open'	New requests default to Open status.
	Check	status in (Open, In Progress, Resolved)	Status must be a valid value.

2.3 Functional Dependencies

The following functional dependencies justify the normalized design. The primary key of each relation determines every non-key attribute of that relation, and no non-key attribute determines another non-key attribute.

- owner_id → name, phone, email, address
- property_id → owner_id, address, type, rent_amount
- tenant_id → name, phone, email, id_proof
- lease_id → property_id, tenant_id, start_date, end_date, deposit
- payment_id → lease_id, amount, payment_date, mode
- request_id → property_id, tenant_id, issue_desc, status

2.4 Normalization up to Third Normal Form (3NF)

Because every determinant listed above is a primary key, all six relations satisfy 1NF, 2NF, 3NF and BCNF [1]. Table 7 records the status of each relation.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
owner	Yes	Yes	Yes	Yes	All attributes depend only on owner_id; no decomposition needed.
property	Yes	Yes	Yes	Yes	owner_id is a foreign key; no transitive dependency among non-key attributes.
tenant	Yes	Yes	Yes	Yes	All attributes depend only on tenant_id.
lease	Yes	Yes	Yes	Yes	A surrogate lease_id is used, so property_id and tenant_id are foreign keys, not partial keys.
payment	Yes	Yes	Yes	Yes	All attributes depend only on payment_id.
maintenance	Yes	Yes	Yes	Yes	All attributes depend only on request_id.

2.5 Data Dictionary

Table 8 gives the data dictionary for all six tables.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
owner	owner_id	INT	PK, AUTO_INCREMENT	Unique owner identifier.
	name	VARCHAR(100)	NOT NULL	Owner's full name.
	phone	VARCHAR(15)	NOT NULL	Owner's contact number.
	email	VARCHAR(100)	-	Owner's email address.
	address	VARCHAR(200)	-	Owner's mailing address.
property	property_id	INT	PK, AUTO_INCREMENT	Unique property identifier.
	owner_id	INT	FK → owner.owner_id, NOT NULL	Owner of the property.
	address	VARCHAR(200)	NOT NULL	Property address.
	type	VARCHAR(20)	NOT NULL, CHECK	Apartment / House / Shop / Office.
	rent_amount	DECIMAL(10,2)	NOT NULL, CHECK > 0	Monthly rent amount.
tenant	tenant_id	INT	PK, AUTO_INCREMENT	Unique tenant identifier.
	name	VARCHAR(100)	NOT NULL	Tenant's full name.
	phone	VARCHAR(15)	NOT NULL	Tenant's contact number.
	email	VARCHAR(100)	-	Tenant's email address.
	id_proof	VARCHAR(50)	-	Identification document reference.
lease	lease_id	INT	PK, AUTO_INCREMENT	Unique lease identifier.
	property_id	INT	FK → property.property_id, NOT NULL	Property covered by the lease.
	tenant_id	INT	FK → tenant.tenant_id, NOT NULL	Tenant holding the lease.
	start_date	DATE	NOT NULL	Lease start date.
	end_date	DATE	NOT NULL, CHECK > start_date	Lease end date.
	deposit	DECIMAL(10,2)	CHECK > 0	Security deposit amount.
payment	payment_id	INT	PK, AUTO_INCREMENT	Unique payment identifier.
	lease_id	INT	FK → lease.lease_id, NOT NULL	Lease the payment applies to.
	amount	DECIMAL(10,2)	NOT NULL, CHECK > 0	Amount paid.
	payment_date	DATE	NOT NULL	Date payment was made.
	mode	VARCHAR(20)	NOT NULL, CHECK	Cash / UPI / Bank Transfer / Cheque.

Table	Attribute	Data Type	Key / Constraint	Description
maintenance	request_id	INT	PK, AUTO_INCREMENT	Unique request identifier.
	property_id	INT	FK → property.property_id, NOT NULL	Property the request concerns.
	tenant_id	INT	FK → tenant.tenant_id, NOT NULL	Tenant who raised the request.
	issue_desc	VARCHAR(255)	NOT NULL	Description of the issue.
	status	VARCHAR(20)	NOT NULL, DEFAULT 'Open'	Open / In Progress / Resolved.

2.6 Implementation of Logical Design: DDL and Sample Data

The database was implemented in MySQL. Table 9 summarizes the implementation environment. Representative CREATE TABLE statements for two of the six tables and sample DML statements are shown below; the complete script is attached as Appendix A.

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL
DBMS Version	8.0 (script also verified on MariaDB 10.11)
Number of Tables	6 (owner, property, tenant, lease, payment, maintenance)
Sample Data	3 owners, 5 properties, 4 tenants, 4 leases, 4 payments and 3 maintenance requests, used for testing and for the figures in this report.

```
CREATE TABLE owner (
  owner_id INT AUTO_INCREMENT PRIMARY KEY,
  name     VARCHAR(100) NOT NULL,
  phone    VARCHAR(15)  NOT NULL,
  email    VARCHAR(100),
  address  VARCHAR(200)
);

CREATE TABLE lease (
  lease_id    INT AUTO_INCREMENT PRIMARY KEY,
  property_id INT NOT NULL,
  tenant_id   INT NOT NULL,
  start_date  DATE NOT NULL,
  end_date    DATE NOT NULL,
  deposit     DECIMAL(10,2) CHECK (deposit > 0),
  CHECK (end_date > start_date),
  FOREIGN KEY (property_id) REFERENCES property(property_id),
  FOREIGN KEY (tenant_id)   REFERENCES tenant(tenant_id)
);

INSERT INTO owner (name,phone,email,address)
VALUES ('Ramesh Gupta','9876500001',
        'ramesh.gupta@example.com','Banjara Hills, Hyderabad');

INSERT INTO property (owner_id,address,type,rent_amount)
VALUES (1,'Flat 301, Lake View Apartments, Banjara Hills',
        'Apartment',25000.00);
```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The Property Rental and Lease Management application is a Python-based web application used by a single property-manager role. It provides a dashboard summarizing occupancy, revenue and overdue rent, and one screen per module (Owners, Properties, Tenants, Leases, Payments, Maintenance) for adding, searching, editing and removing records. Every screen reads from and writes to the MySQL database described in Chapter 2; no data is held only in the application. The business rules defined in Section 1.4 are enforced in the application layer before a write is attempted and again by the database constraints. An interactive front-end prototype of the same design is referred to in Appendix C.

3.2 Development Environment and Database Connectivity

The development environment and the database connectivity settings are given in Table 10. The application opens a connection through the mysql-connector-python driver [3] for each request, with the host, user, password and database name read from environment variables, and serves HTML pages through the Flask framework [4].

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.x
DBMS	MySQL 8.0
IDE / Framework / GUI Toolkit	Flask (web framework) with Jinja2 templates; HTML/CSS user interface
Database Connector / Driver	mysql-connector-python
Operating Environment	Cross-platform; developed and tested on Linux (Ubuntu) with Python 3.12
Repository Link	[Add GitHub repository link]

3.3 Application Modules and Database Mapping

Each module of the application is mapped to the tables it uses in Table 11.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard	Shows occupancy, revenue and overdue-rent summary.	property, lease, payment, maintenance, tenant	Read (aggregate)
Owners	Manage property owners.	owner	Create, Read, Search, Update, Delete

Module / Screen	Purpose	Tables Used	Main Operations
Properties	Manage properties; show computed occupancy status.	property, owner, lease	Create, Read, Search, Update, Delete
Tenants	Manage tenants.	tenant	Create, Read, Search, Update, Delete
Leases	Create leases; validate date range; show rent-due status.	lease, property, tenant, payment	Create, Read, Search, Update, Delete
Payments	Record rent payments against a lease.	payment, lease, tenant	Create, Read, Search, Update, Delete
Maintenance	Log and track maintenance requests.	maintenance, property, tenant	Create, Read, Search, Update, Delete

3.4 CRUD Operations

Table 12 shows how each CRUD operation is performed in the application and where evidence of it appears in Section 3.7.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	"+ New" form on every module screen.	All six tables	Figures 3 and 6
Read / Search	Module listings, search box and dashboard summary.	All six tables	Figures 2 and 4
Update	"Edit" action pre-fills the form; "Update" saves the change.	All six tables	Figure 5
Delete / Controlled Removal	"Remove" action with confirmation, blocked when the record is still referenced elsewhere.	All six tables	Figure 8

3.5 Business Rules, Validation and Error Handling

Every write operation uses parameterized queries (never string-concatenated SQL) to prevent SQL injection, and each database write is wrapped in a try/except block that rolls back and reports the error instead of leaving the application in an inconsistent state. Destructive actions ask for confirmation. Table 13 lists the main rules and how they are enforced.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Lease end date must be after the start date.	Checked in the application before INSERT or UPDATE; the CHECK constraint is a second line of defence.	"End date must be after the start date."
Required fields cannot be empty.	HTML required attribute plus server-side validation.	"Name: This field is required."

Rule / Validation	How Implemented	Evidence / Message
A record cannot be deleted while referenced elsewhere.	The application counts matching foreign-key references before DELETE; the database's FOREIGN KEY constraint is a second line of defence.	"Can't remove this record - it is referenced by a property. Remove that first."
Positive amounts only (rent, deposit, payment).	Input validation in the application, mirrored by CHECK constraints in the database.	"Amount (INR) must be a positive value."
Database errors must not crash the application.	try/except around every write with rollback; the MySQL message is shown to the user.	"Database rejected the record: ..."
Destructive actions need confirmation.	Browser confirmation dialog on the Remove button.	"Remove this record?"

3.6 Reports and Database Operations

The application generates the reports in Table 14 directly from the database; their outputs appear on the dashboard (Figure 2) and in the module listings.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Occupancy by property type	GROUP BY with COUNT() and a conditional SUM(EXISTS(...))	Table of property type, total units and occupied units on the dashboard.
Monthly revenue	SUM() over payment.amount filtered by MONTH() and YEAR() of payment_date	Single revenue figure shown on the dashboard.
Overdue rent payments	Subquery with NOT IN, joined across lease, tenant and property	List of tenants and properties with no payment recorded for the current month.
Property occupancy status	Correlated EXISTS subquery on active leases	Occupied / Vacant column in the Properties listing.
Lease rent status	Correlated EXISTS subquery on payments of the current month	Paid / Due / Not active column in the Leases listing.

3.7 Application Screens

The screens below demonstrate the application running against the MySQL database: a report/output (Figure 2), data entry (Figure 3), search (Figure 4), update (Figure 5), a project-specific transaction (Figure 6) and validation and integrity checks (Figures 7 and 8).

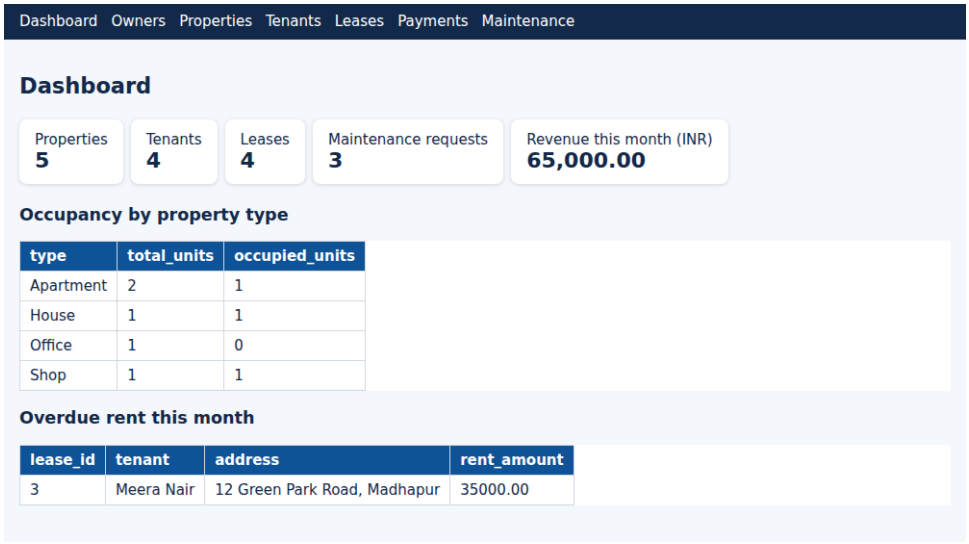


Figure 2. Dashboard: Occupancy, Revenue and Overdue-Rent Reports

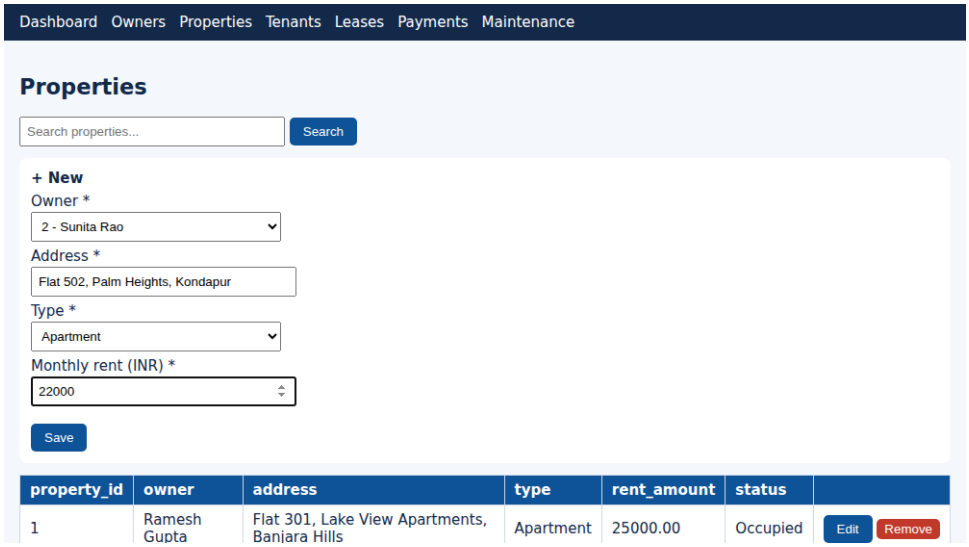


Figure 3. Add Property: Data Entry Screen

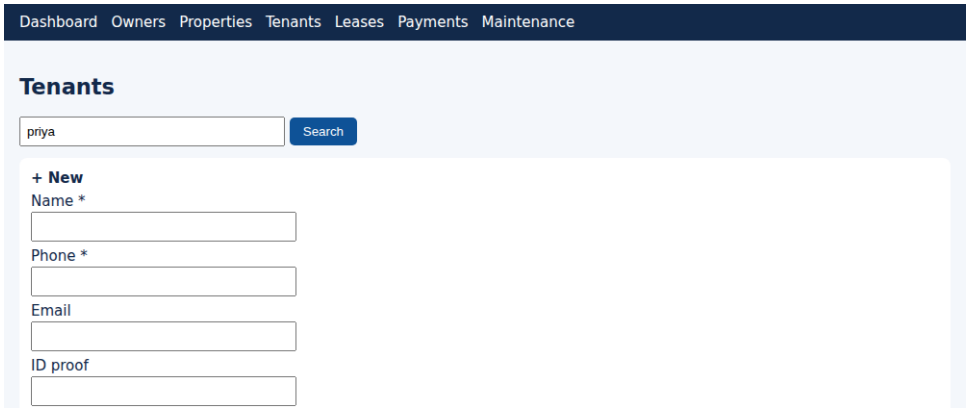


Figure 4. Search: Tenant Search Results

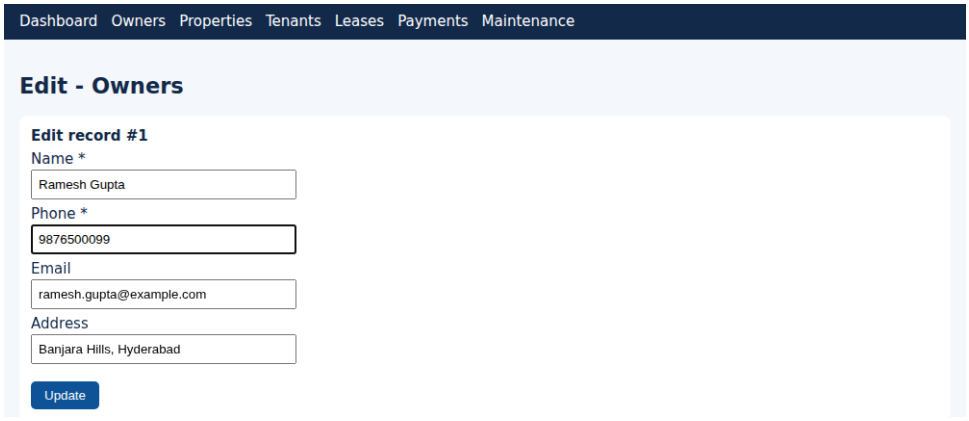


Figure 5. Edit Owner: Update Operation

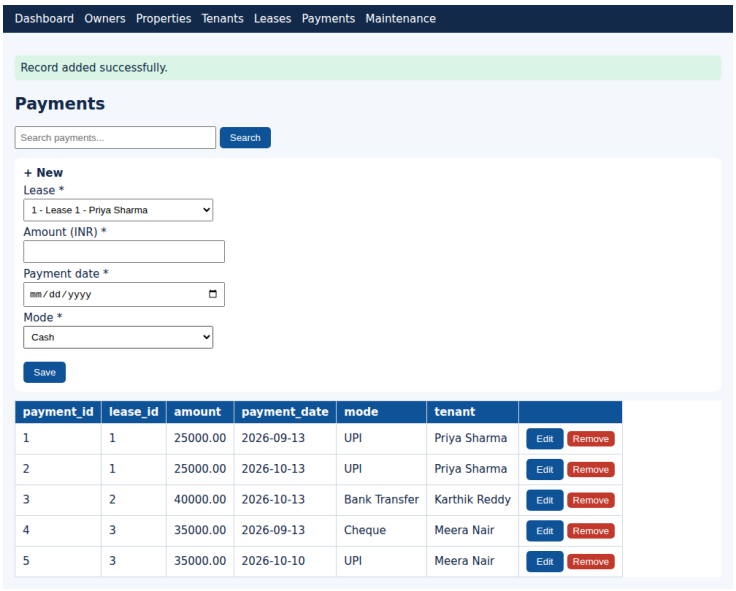


Figure 6. Rent Payment Transaction

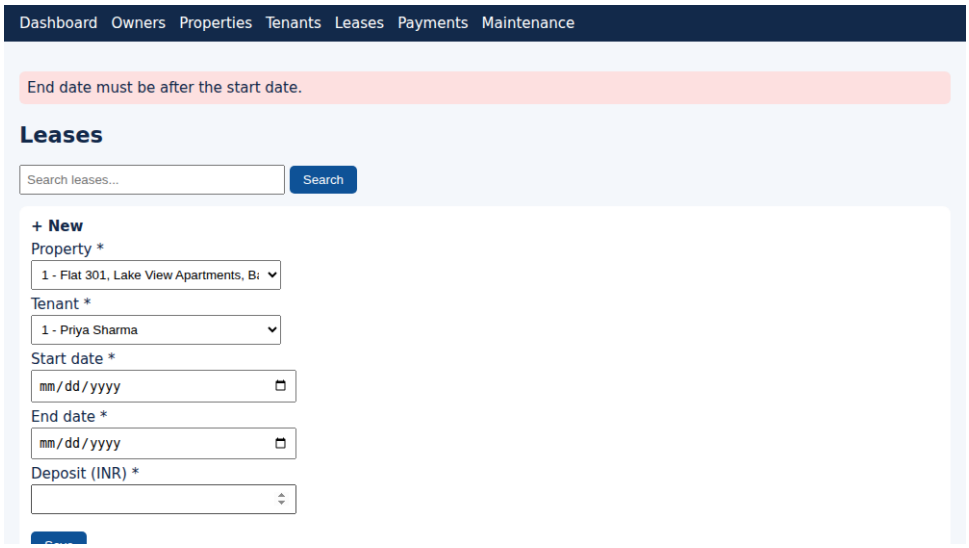


Figure 7. Create Lease: Validation of Lease Dates

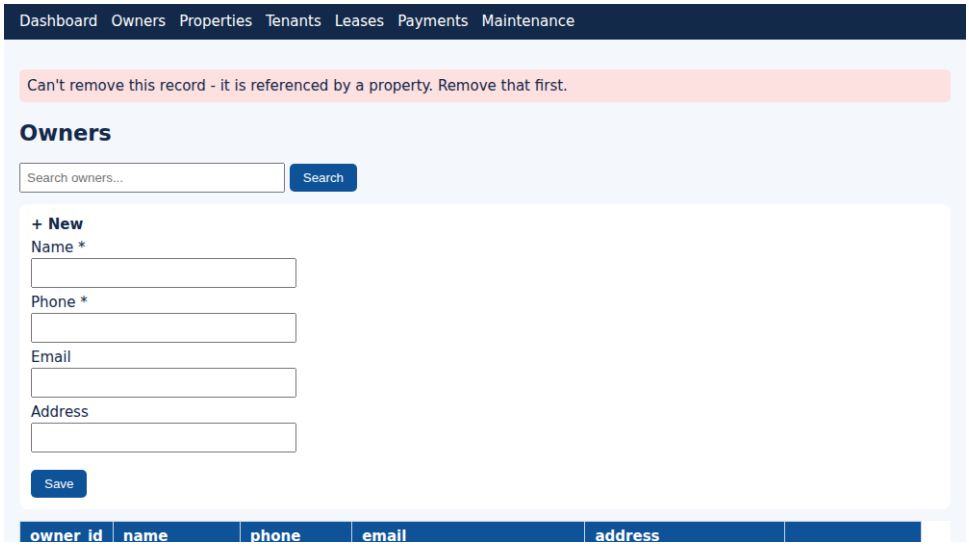


Figure 8. Referential Integrity: Blocked Deletion

3.8 Testing and Results

Table 15 lists the positive and negative test cases executed on the running application.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Add property	Add a new property with valid data.	Record inserted and visible in the listing.	Record inserted and visible in the Properties listing (Figure 3).	Pass
TC-02	Lease date validation	Create a lease with end date before start date.	System rejects the lease with a validation error.	Rejected with "End date must be after the start date." (Figure 7).	Pass
TC-03	Record payment	Record a rent payment for an active lease with no payment this month.	Payment saved; rent status changes to Paid and the lease leaves the overdue list.	Payment saved; status became Paid and the overdue list was empty (Figure 6).	Pass
TC-04	Maintenance request	Log a maintenance request for a property.	Request logged with status 'Open'.	Request saved with status 'Open'.	Pass
TC-05	Referential integrity	Remove an owner who has a property.	Deletion is blocked with an explanatory message.	Blocked with the message shown in Figure 8.	Pass
TC-06	Occupancy report	Open the dashboard occupancy-by-type report.	Correct counts per property type.	Apartment 2/1, House 1/1, Office 1/0, Shop 1/1 (total/occupied), matching a manual check.	Pass
TC-07	Required fields	Submit the Add Tenant form with the name empty.	Form rejects the submission.	Rejected with "Name: This field is required."	Pass
TC-08	Update record	Change an owner's phone number and save.	Change is stored and shown in the listing.	Phone number updated successfully (Figure 5).	Pass

All eight test cases passed. The results confirm that the application reads from and writes to the designed database, that foreign-key relationships are respected, and that the key business rules (valid lease date

ranges, required fields, positive amounts and referential integrity on deletion) are enforced consistently. Search and negative-amount checks were also exercised during testing.

CHAPTER 4

CONCLUSION

This project addressed the problem of manual, error-prone rental record-keeping by designing a normalized relational database for a Property Rental and Lease Management System and implementing a Python application on top of it. The database design was carried from an approved ER model to a relational schema normalized to 3NF, with primary keys, foreign keys and check constraints implemented in MySQL. The application provides working create, read, search, update and controlled delete operations across all six modules, enforces the core business rules, and produces occupancy, revenue and overdue-payment reports directly from the database.

Testing against eight positive and negative test cases confirmed that the application behaves correctly for valid and invalid inputs. The completed system demonstrates, in a small but complete form, how a normalized database design and an application built on top of it can replace an error-prone manual process with a consistent, rule-enforcing alternative. The project also strengthened the understanding of ER modelling, normalization, SQL constraints and database connectivity.

4.1 Limitations and Future Enhancements

- The application is a single-user system with no authentication or role-based access control; a production version would need separate owner, tenant and administrator logins.
- Rent payments are recorded manually; integrating an online payment gateway would let tenants pay directly and automate the payment record.
- The reporting module is limited to three reports; future versions could add lease-expiry alerts and year-over-year revenue trends.
- Deletion is restricted rather than archived; a soft-delete or audit trail would preserve history of removed records.

REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2017.

[2] Oracle Corporation, "MySQL 8.0 Reference Manual," MySQL. Accessed: Oct. 7, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[3] Oracle Corporation, "MySQL Connector/Python Developer Guide," MySQL. Accessed: Oct. 7, 2026. [Online]. Available: https://dev.mysql.com/doc/connector-python/en/

[4] Pallets Projects, "Flask Documentation," Flask. Accessed: Oct. 7, 2026. [Online]. Available: https://flask.palletsprojects.com/

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete database script (DDL for all six tables with constraints, sample INSERT statements, the report queries and constraint demonstrations) is submitted as the file property_rental_schema.sql in the project repository. It is executed with: mysql -u root -p < property_rental_schema.sql

APPENDIX B - APPLICATION SOURCE CODE

The complete application source code is available in the project repository: [Add GitHub repository link]. The repository contains app.py (Flask application), requirements.txt, property_rental_schema.sql, the interactive prototype (prototype/index.html), the review presentations and the screenshots.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Figure C1 shows the dashboard after the payment of Figure 6 was recorded: the overdue-rent list is now empty and the revenue figure has increased. In addition, an interactive front-end prototype of the same six-table design, "Reddy's Estate", with a dashboard, search, edit and an in-page schema/SQL view, is available at https://claude.ai/artifact/A2EtJFV7ZV2f5vRn9ARLnY and as prototype/index.html in the repository.

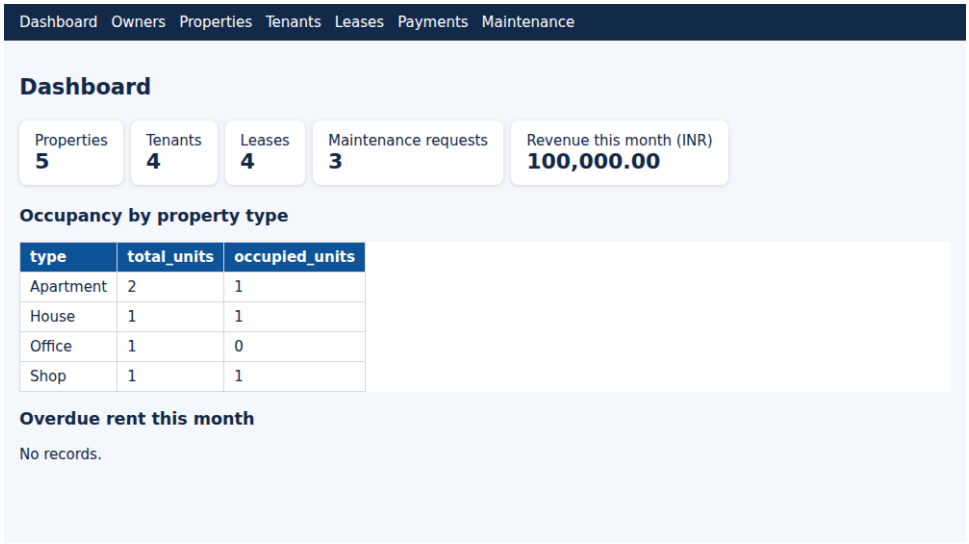


Figure C1. Dashboard after Recording the Payment